

CSCI 650: Applied Machine Learning

Sai Rikwith Daggu [012147396]

Assignment#2-

Introduction- This project investigates image processing methods and Convolutional Neural Networks (CNNs) for classifying images. In Part 1, we applied image filtering techniques like edge detection, blurring, and scaling to prepare the images. In Part 2, we built and trained a CNN model with the CIFAR-10 dataset to categorize images into 10 distinct classes. The project emphasizes feature extraction through image processing and how CNNs learn to recognize hierarchical features to achieve precise classification.

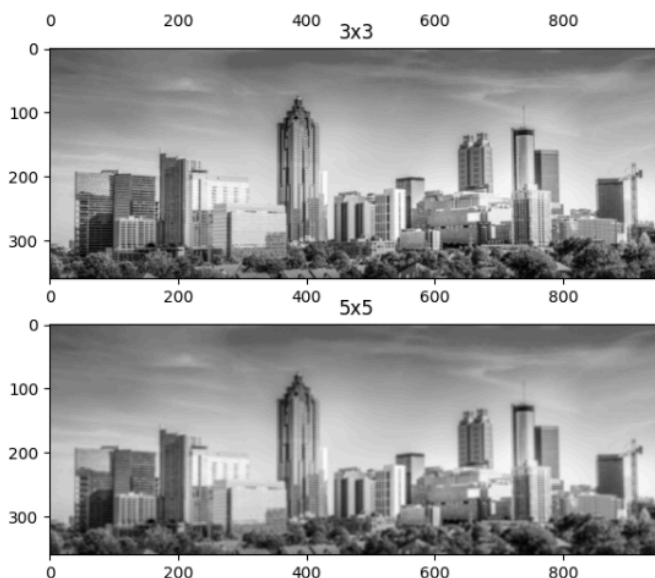
Part 1: Image Filtering:

1. Blurring:

```
# 3x3 averaging kernel
S3x3 = np.ones((3, 3), dtype=np.float32) / 9 # 3x3 averaging kernel
blurred_image3 = cv2.filter2D(gray, -1, S3x3)
fig.add_subplot(4,1,3)
plt.imshow(blurred_image3, cmap='gray')
plt.title('3x3')

# 5x5 averaging kernel
S5x5 = np.ones((5, 5), dtype=np.float32) / 25 # 5x5 averaging kernel
blurred_image5 = cv2.filter2D(gray, -1, S5x5)
fig.add_subplot(4,1,4)
plt.imshow(blurred_image5, cmap='gray')
plt.title('5x5')

plt.show()
```



2. Edge Detection:

```
# Sobel Edge Detection

# Sobel kernels for detecting vertical and horizontal edges
Kx = np.array([[-1, 0, 1],
               [-2, 0, 2],
               [-1, 0, 1]]) # Vertical edge detection

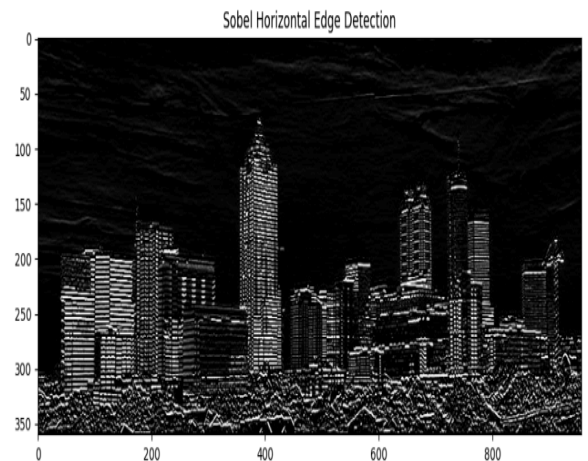
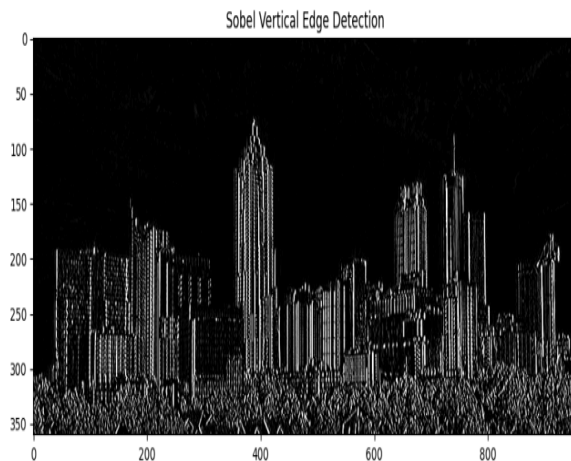
Ky = np.array([[-1, -2, -1],
               [0, 0, 0],
               [1, 2, 1]]) # Horizontal edge detection

# Applying the Sobel kernels using filter2D
sobel_vertical = cv2.filter2D(gray, -1, Kx)
sobel_horizontal = cv2.filter2D(gray, -1, Ky)

# Display the results of Sobel edge detection
fig = plt.figure(figsize=(24, 12))
fig.add_subplot(1, 2, 1)
plt.imshow(sobel_vertical, cmap='gray')
plt.title('Sobel Vertical Edge Detection')

fig.add_subplot(1, 2, 2)
plt.imshow(sobel_horizontal, cmap='gray')
plt.title('Sobel Horizontal Edge Detection')

plt.show()
```



3. Corner Detection:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image and convert to grayscale
image = cv2.imread('building2.jpg', cv2.IMREAD_GRAYSCALE)

# Define a Corner Detection Kernel
corner_kernel = np.array([
    [-2, -1, 0],
    [-1, 1, 1],
    [ 0, 1, 2]
])

# Apply convolution
corner_detection = cv2.filter2D(image, -1, corner_kernel)

# Apply Adaptive Threshold for clearer black and white image
_, binary_output = cv2.threshold(corner_detection, 30, 255, cv2.THRESH_BINARY_INV)

# Display the result
plt.figure(figsize=(10,6))
plt.imshow(binary_output, cmap='gray', vmin=0, vmax=255)
plt.title('Convolution-Based Corner Detection')
plt.axis('off')
plt.show()
```

Convolution-Based Corner Detection



4. Scaling:

```
# Scaling (2x2 and 4x4 scaling)

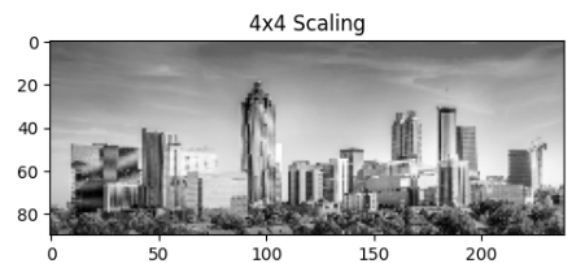
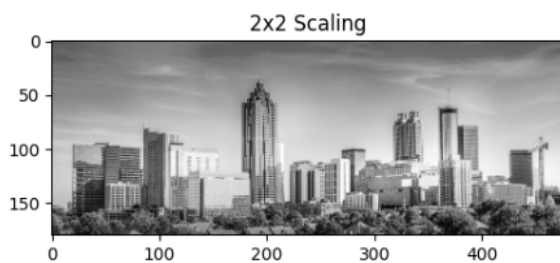
# 2x2 scaling (downsampling)
scaled_2x2 = cv2.resize(gray, (gray.shape[1] // 2, gray.shape[0] // 2), interpolation=cv2.INTER_LINEAR)

# 4x4 scaling (downsampling)
scaled_4x4 = cv2.resize(gray, (gray.shape[1] // 4, gray.shape[0] // 4), interpolation=cv2.INTER_LINEAR)

# Display the results of scaling
fig = plt.figure(figsize=(12, 6))
fig.add_subplot(1, 2, 1)
plt.imshow(scaled_2x2, cmap='gray')
plt.title('2x2 Scaling')

fig.add_subplot(1, 2, 2)
plt.imshow(scaled_4x4, cmap='gray')
plt.title('4x4 Scaling')

plt.show()
```



Part 2: CNN Implementation:

Dataset & Preprocessing:

The CIFAR-10 dataset consists of 60,000 labeled images, split into 50,000 training images and 10,000 test images across 10 categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each image is 32×32 pixels in size with three color channels (RGB), making it a standard benchmark for image classification tasks. To improve model performance, we applied data normalization, scaling pixel values to the range of -1 to 1, ensuring all features contribute equally to the learning process. This helps mitigate issues caused by varying pixel intensities and speeds up convergence.

Training & Evaluation:

We trained our CNN model using the gradient descent optimization method to minimize classification error over several epochs. The training was conducted with the following hyperparameters:

- Loss Function: CrossEntropyLoss
- Optimizer: SGD with momentum (0.9)
- Batch Size: 64

- Epochs: 10
- Learning Rate: 0.01

During training, the model's weights were updated after each epoch to reduce the loss.

1. Defining Network Architecture:

```
import torch.nn as nn
import torch.nn.functional as F

# define the CNN architecture
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()

        # Build multiple convolutional layers
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1) # 32x32x3 -> 32x32x32
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1) # 32x32x32 -> 32x32x64
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1) # 32x32x64 -> 32x32x128

        # max pooling layer
        self.pool = nn.MaxPool2d(2, 2) # Reduces dimensions by half

        # Fully connected layers (linear layers)
        self.fc1 = nn.Linear(128 * 4 * 4, 512) # after 2 maxpool layers, image size is 8x8 -> 128*8*8
        self.fc2 = nn.Linear(512, 10) # Output 10 classes (CIFAR-10 has 10 classes)

        # Dropout layer to avoid overfitting
        self.dropout = nn.Dropout(0.25) # Dropout with probability 0.25

    def forward(self, x):
        # Apply convolutional layers with ReLU activation and max pooling
        x = self.pool(F.relu(self.conv1(x))) # 32x32x3 -> 16x16x32
        x = self.pool(F.relu(self.conv2(x))) # 16x16x32 -> 8x8x64
        x = self.pool(F.relu(self.conv3(x))) # 8x8x64 -> 4x4x128

        # Flatten the tensor to feed into fully connected layers
        x = x.view(-1, 128 * 4 * 4) # Flatten (batch_size, 128*4*4)

        # Apply dropout for regularization
        x = self.dropout(x)

        # Fully connected layers with ReLU activation
        x = F.relu(self.fc1(x)) # First hidden layer
        x = self.dropout(x) # Optional dropout
        x = self.fc2(x) # Output layer (no activation for raw scores)

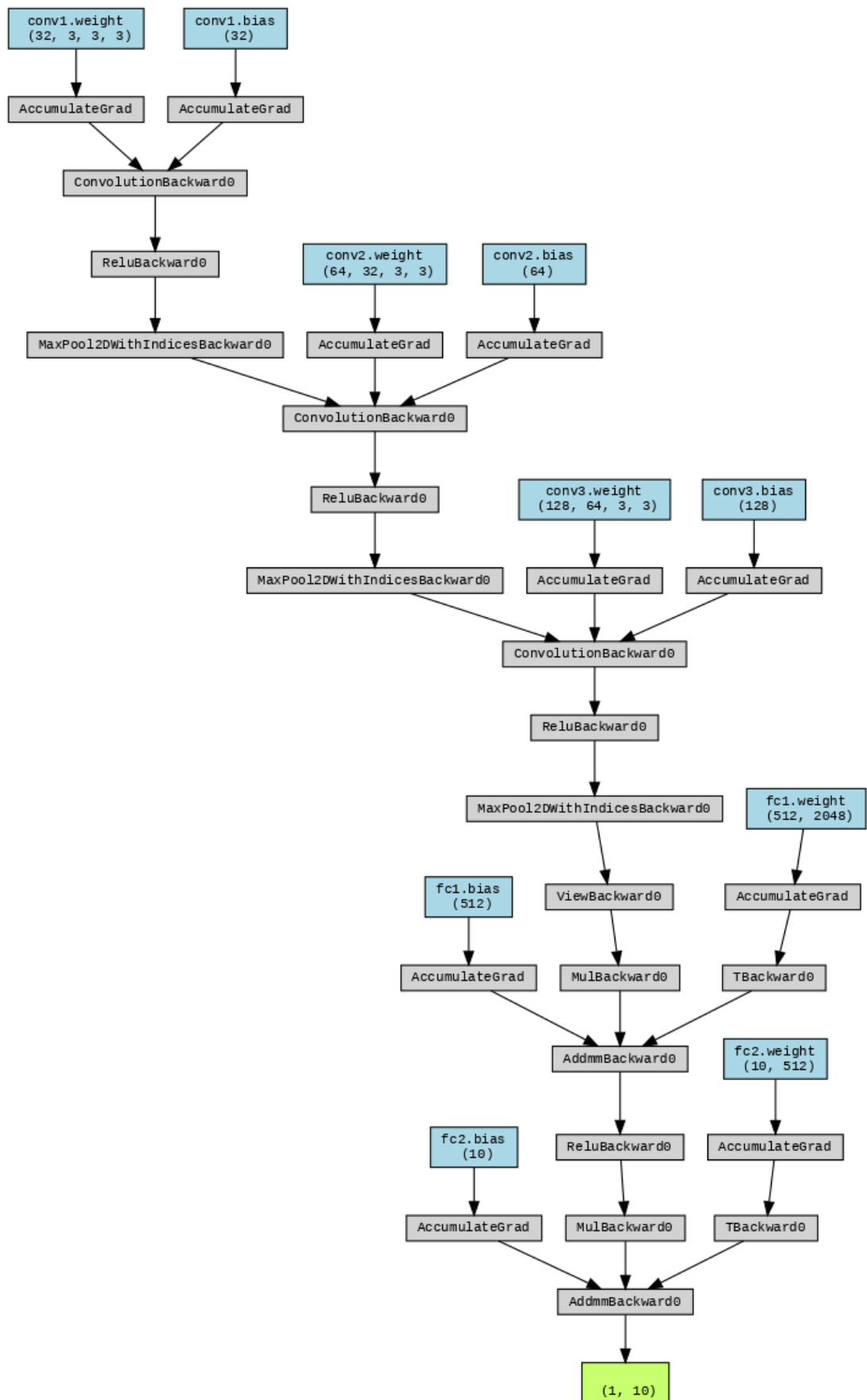
        return x

# create a complete CNN model
model = Net()
print(model)

# move tensors to GPU if CUDA is available
if train_on_gpu:
    model.cuda()
```

```
Net(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=2048, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=10, bias=True)
  (dropout): Dropout(p=0.25, inplace=False)
)
```

2. graphical illustration of CNN architecture:



3. Specify Loss Function and Optimizer:

```
import torch.optim as optim

# specify loss function (categorical cross-entropy)
criterion = nn.CrossEntropyLoss()

# specify optimizer (SGD)
optimizer_sgd = optim.SGD(model.parameters(), lr=0.01)

# specify optimizer (ADAM)
optimizer_adam = optim.Adam(model.parameters(), lr=0.001) # ADAM typically works well with smaller learning rates

# Print both optimizers' details
print("SGD Optimizer:", optimizer_sgd)
print("\nADAM Optimizer:", optimizer_adam)

# Select the optimizer you want to use
optimizer = optimizer_adam # Change this to optimizer_sgd to use SGD instead of Adam
```

```
SGD Optimizer: SGD (
  Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.01
    maximize: False
    momentum: 0
    nesterov: False
    weight_decay: 0
)
```

```
ADAM Optimizer: Adam (
  Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.001
    maximize: False
    weight_decay: 0
)
```

4. Train the Network:

```
Epoch: 1      Training Loss: 0.620242      Validation Loss: 0.765456
Validation loss decreased (inf --> 0.765456). Saving model ...
Epoch: 2      Training Loss: 0.569042      Validation Loss: 0.719528
Validation loss decreased (0.765456 --> 0.719528). Saving model ...
Epoch: 3      Training Loss: 0.524880      Validation Loss: 0.725443
Epoch: 4      Training Loss: 0.490299      Validation Loss: 0.732449
Epoch: 5      Training Loss: 0.457411      Validation Loss: 0.761610
```

5. Test the Trained Network:

Test Loss: 0.721776

```
Test Accuracy of airplane: 83% (833/1000)
Test Accuracy of automobile: 87% (878/1000)
Test Accuracy of bird: 63% (637/1000)
Test Accuracy of cat: 59% (595/1000)
Test Accuracy of deer: 74% (748/1000)
Test Accuracy of dog: 56% (568/1000)
Test Accuracy of frog: 84% (847/1000)
Test Accuracy of horse: 78% (783/1000)
Test Accuracy of ship: 84% (843/1000)
Test Accuracy of truck: 78% (787/1000)
```

Test Accuracy (Overall): 75% (7519/10000)

6. Visualize Sample Test Results:



Summary of Findings

- CNN achieved **75% test accuracy** on CIFAR-10.